Lab 7 Policy-Based Reinforcement Learning

314551022 廖洛玄

Introduction

我在這個 Lab 主要是要學會 implement 兩個重要的 Policy-Based RL：A2C (Advantage Actor-Critic) 和 PPO (Proximal Policy Optimization)。我們會在三個不同的任務上測試這些算法的效果：Task 1 用 A2C 解決 Pendulum-v1，Task 2 用 PPO 解決同樣的 Pendulum 環境，Task 3 則是把 PPO 應用到更複雜的 Walker2d-v4 locomotion task。重點在於比較 A2C 和 PPO 的 sample efficiency 和 training stability，特別是 PPO 的 clipped objective 和 GAE (Generalized Advantage Estimation) 如何改善訓練效果。我也會分析 clipping parameter 和 entropy coefficient 等關鍵參數對模型表現的影響。我在這個 Lab 也有用 LLM 來幫我 debug 和找尋實驗結果的原因和怎麼改善我的 code。
!!!!Evaluation 有在 code 資料夾提供三題的 shell script 請助教在 LAB7_314551022_Code 資料夾中執行三個 shell script 如果要換參數要去 shell script 裡換，或是改 test_model_task{number}.py 的 default value。!!!!!

Implementation Details

Stochastic policy gradient and the TD error for A2C

在 A2C 中，我實現了 stochastic policy gradient 和 TD error 的計算。Stochastic Policy Gradient，我用 Gaussian policy 來處理連續動作空間。在 Actor 中，網路輸出 mean 和 std，然後用 Normal(mean, std) 建立 distribution：

```python
def forward(self, state: torch.Tensor):
    x = F.relu(self.fc1(state))
    x = F.relu(self.fc2(x))
    mean = self.mean_layer(x)
    std = F.softplus(self.std_layer(x)) + 1e-6

    dist = Normal(mean, std)
    raw_action = dist.rsample()
    squashed_action = torch.tanh(raw_action)

    log_prob = dist.log_prob(raw_action) - torch.log(
        1 - squashed_action.pow(2) + 1e-6
    )
    log_prob = log_prob.sum(dim=-1)

    # scale to env action space [-2, 2]
    action = squashed_action * 2.0

    # 計算 entropy
    entropy = dist.entropy().sum(dim=-1)

    return action, log_prob, entropy
```

TD Error: TD error 就是 δ = r + γ V(s') - V(s)，我在 update_model 函數中計算：

```python
# 計算 target value
with torch.no_grad():
    next_value = self.critic(next_state)
    bootstrap_mask = 0.0 if terminated else 1.0
    bootstrap_mask = torch.as_tensor(bootstrap_mask, dtype=torch.float32, device=self.device)
    target = reward + self.gamma * next_value * bootstrap_mask

# Critic loss
value = self.critic(state)
value_loss = F.mse_loss(value, target)

# 更新 Critic
self.critic_optimizer.zero_grad()
value_loss.backward()
self.critic_optimizer.step()

# 計算 advantage
advantage = (target - value).detach()

# Policy loss (使用正確的 entropy)
policy_loss = -(log_prob * advantage + self.entropy_weight * entropy)
```

Policy loss 用 advantage 作為 baseline： policy_loss = -(log_prob * advantage + self.entropy_weight * entropy)

Clipped objective in PPO

我的 PPO 核心是 clipped surrogate objective，防止 policy 更新太大導致不穩定。

```python
# Get new action distribution
_, dist = self.actor(state)
log_prob = dist.log_prob(action)
ratio = (log_prob - old_log_prob).exp()

# PPO-Clip objective
surr1 = ratio * adv
surr2 = torch.clamp(ratio, 1.0 - self.epsilon, 1.0 + self.epsilon) * adv
policy_loss = -torch.min(surr1, surr2).mean()

# Add entropy bonus
entropy = dist.entropy().mean()
actor_loss = policy_loss - self.entropy_weight * entropy
```

重點是 ratio = π_θ(a|s) / π_θ_old(a|s) 表示新舊 policy 的比例，然後用 torch.clamp 把它限制在 [1- ε , 1+ ε] 範圍內。最後取 clipped 和 unclipped 的最小值，確保更新不會太激進。

Estimator of GAE

GAE (Generalized Advantage Estimation) 是用來減少 advantage estimation 的 variance。

```python
def compute_gae(
    next_value: list, rewards: list, masks: list, values: list, gamma: float, tau: float) -> List:
    """Compute gae."""

    ############TODO#############
    # Generalized Advantage Estimation (GAE)
    # GAE combines TD errors with exponentially-weighted averaging
    values = values + [next_value]
    gae = 0
    gae_returns = []

    # Iterate backwards through the trajectory
    for step in reversed(range(len(rewards))):
        # Calculate TD error: δ_t = r_t + γ * V(s_{t+1}) - V(s_t)
        delta = rewards[step] + gamma * values[step + 1] * masks[step] - values[step]

        # Update GAE: A_t = δ_t + (γ * τ) * A_{t+1}
        # tau (λ in GAE paper) controls bias-variance tradeoff
        gae = delta + gamma * tau * masks[step] * gae

        # The return is value + advantage
        gae_returns.insert(0, gae + values[step])
    ############################
    return gae_returns
```

這裡 tau (λ) 是 GAE 參數，控制 bias-variance tradeoff。當 λ =1 時等於 Monte Carlo， λ =0 時等於 TD(0)。我設定 τ =0.95 在兩者間取得平衡。

Collect samples from the environment

我用 rollout-based 的方式收集樣本。對於 A2C，每步都立即更新；對於 PPO，我收集一個 rollout (預設 2048 步) 後再批次更新

```
# Collect rollout
for _ in range(self.rollout_len):
    if self.total_step > max_steps:
        break

    self.total_step += 1
    pbar.update(1)

    action = self.select_action(state)
    next_state, reward, done = self.step(action)

    state = next_state
    score += reward[0][0]
    rollout_score += reward[0][0]
```

在 select_action 和 step 函數中，我會自動把 states, actions, values, log_probs, rewards, masks 存到 memory 供後續訓練用。

Enforce exploration (despite that both A2C and PPO are on-policy RL methods)

1. Entropy Regularization: 在 loss 中加入 entropy bonus

```
# PPO-Clip objective
surr1 = ratio * adv
surr2 = torch.clamp(ratio, 1.0 - self.epsilon, 1.0 + self.epsilon) * adv
policy_loss = -torch.min(surr1, surr2).mean()

# Add entropy bonus
entropy = dist.entropy().mean()
actor_loss = policy_loss - self.entropy_weight * entropy

actor_loss.backward()
nn.utils.clip_grad_norm_(self.actor.parameters(), 0.5)
self.actor_optimizer.step()
actor_losses.append(actor_loss.item())
```

2. Gaussian Policy with Learnable Std: 讓網路學習 action distribution 的 standard deviation

3. Proper Network Initialization: 用 orthogonal initialization 讓網路在訓練初期有足夠隨機性

Weight & Bias to track model performance and the loss values (including actor loss, critic loss, and the entropy).

我在 Lab 用到用 wandb 追蹤訓練過程中的各種 metrics：

```
# Get mean and log_std
mean = self.fc_mean(x)
log_std = self.fc_log_std(x)

# Clamp log_std to be within bounds
log_std = torch.clamp(log_std, self.log_std_min, self.log_std_max)

# Add small epsilon to prevent numerical issues
std = torch.exp(log_std)

# Create Normal distribution
dist = Normal(mean, std)

# Sample action (don't apply tanh here, let the agent handle it)
action = dist.rsample()  # Use rsample for reparameterization trick
```

```
wandb.log({
    "episode_reward": score,
    "episode": episode_count,
    "recent_20_avg": recent_avg,
    "environment_steps": self.total_step
})
```

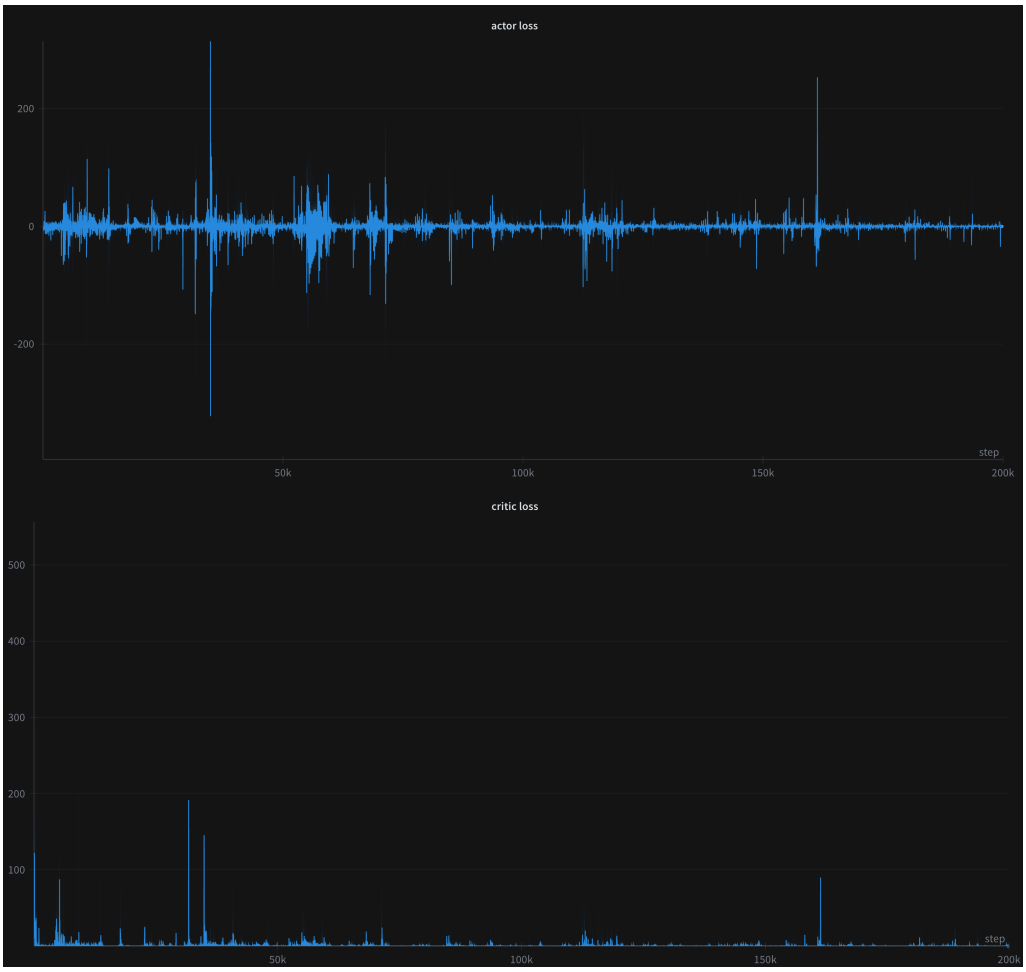
```python
# Log losses to wandb
wandb.log({
    "actor_loss": actor_loss,
    "critic_loss": critic_loss,
    "rollout": rollout_count,
    "total_steps": self.total_step
})
```

最主要的是 Episode Reward: 每個 episode 的總獎勵、Average Reward: 滑動平均獎勵（用來判斷收斂）、Actor/Critic Loss: 監控訓練穩定性、Environment Steps: x 軸用環境步數而不是 episodes 和 episode

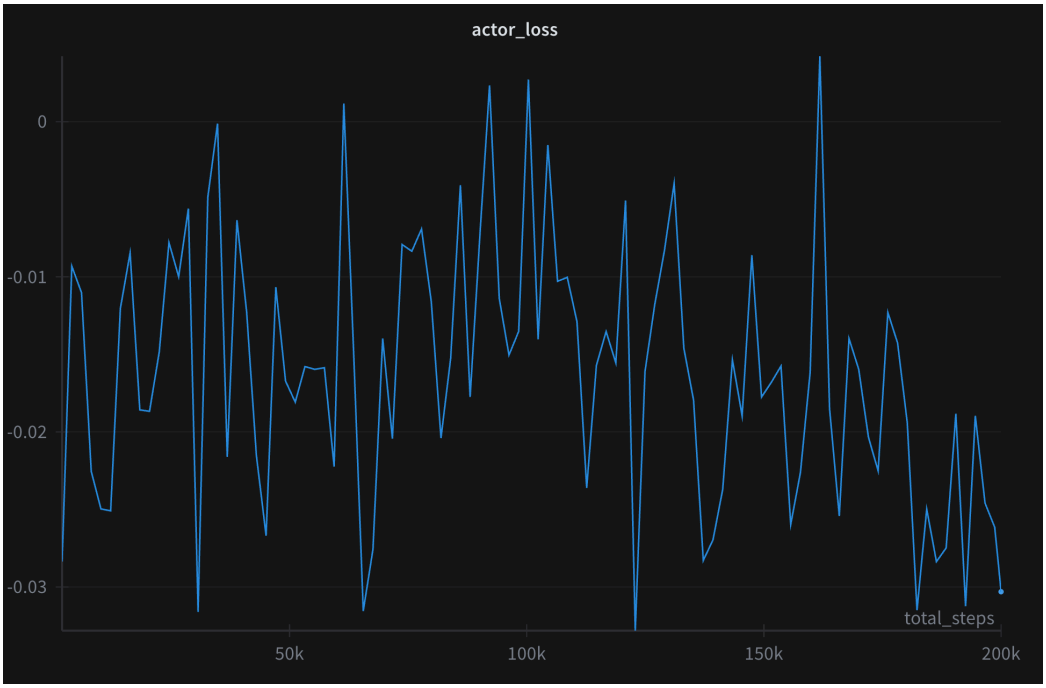
Analysis and discussions

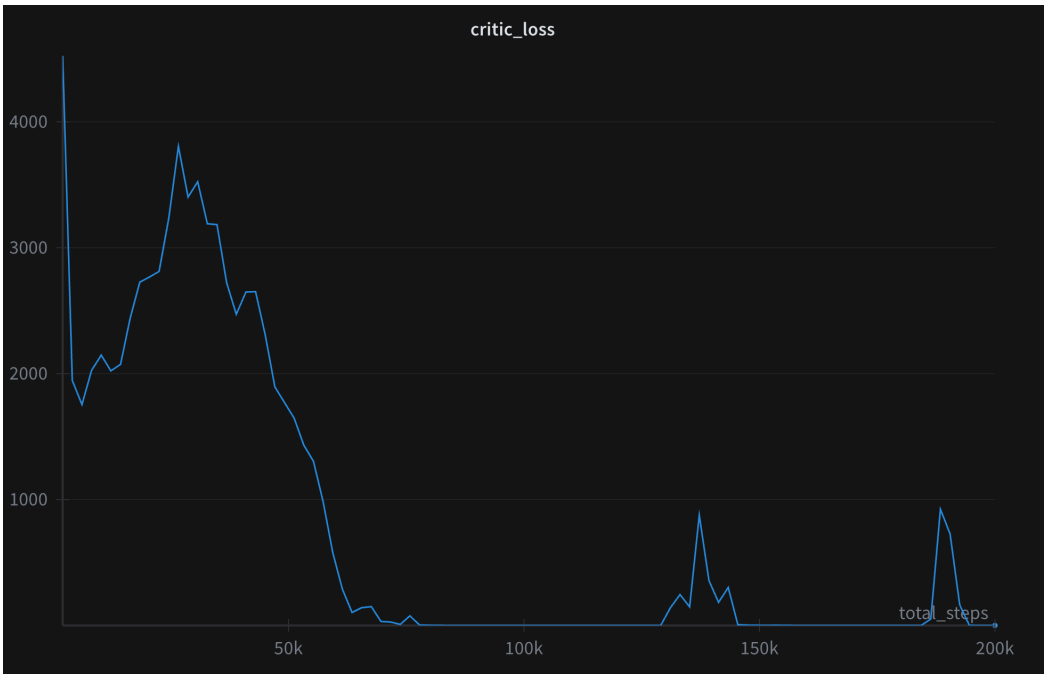
Training curves

從下面的 loss curve 我們可以看到 A2C 的 actor loss 表現出非常不穩定的特徵，我查到是因為 A2C 的設計問題，沒有 policy update 的保護機制，當 advantage 估計不準確時會導致破壞性的更新；PPO 的 curve 在 Pendulum 環境上表現出顯著改善，因為 PPO 的 clipped objective，限制了 policy 更新的幅度，避免了劇烈變化；而 Walker2d 可以看到 PPO 處理複雜環境效果，有個尖峰可能是因為遇到了特別困難的 episode 或是 exploration 過程中的暫時性問題，但 PPO 的 clipping mechanism 讓它快速恢復到正常值。從 critic loss 可以看出，A2C 在 Pendulum 上雖然大部分時間維持低 loss，但會出現偶發性的變動後就快速恢復，因為 online learning 對 outlier 的敏感性。PPO 則看到了一般的收斂，從高 loss 平滑下降並趨於穩定，偶爾因 policy update 引起的週期性調整，顯示更可控的學習過程。在複雜的 Walker2d 環境中，PPO 的 critic loss 維持在較高但穩定的 noise avg。
Task1

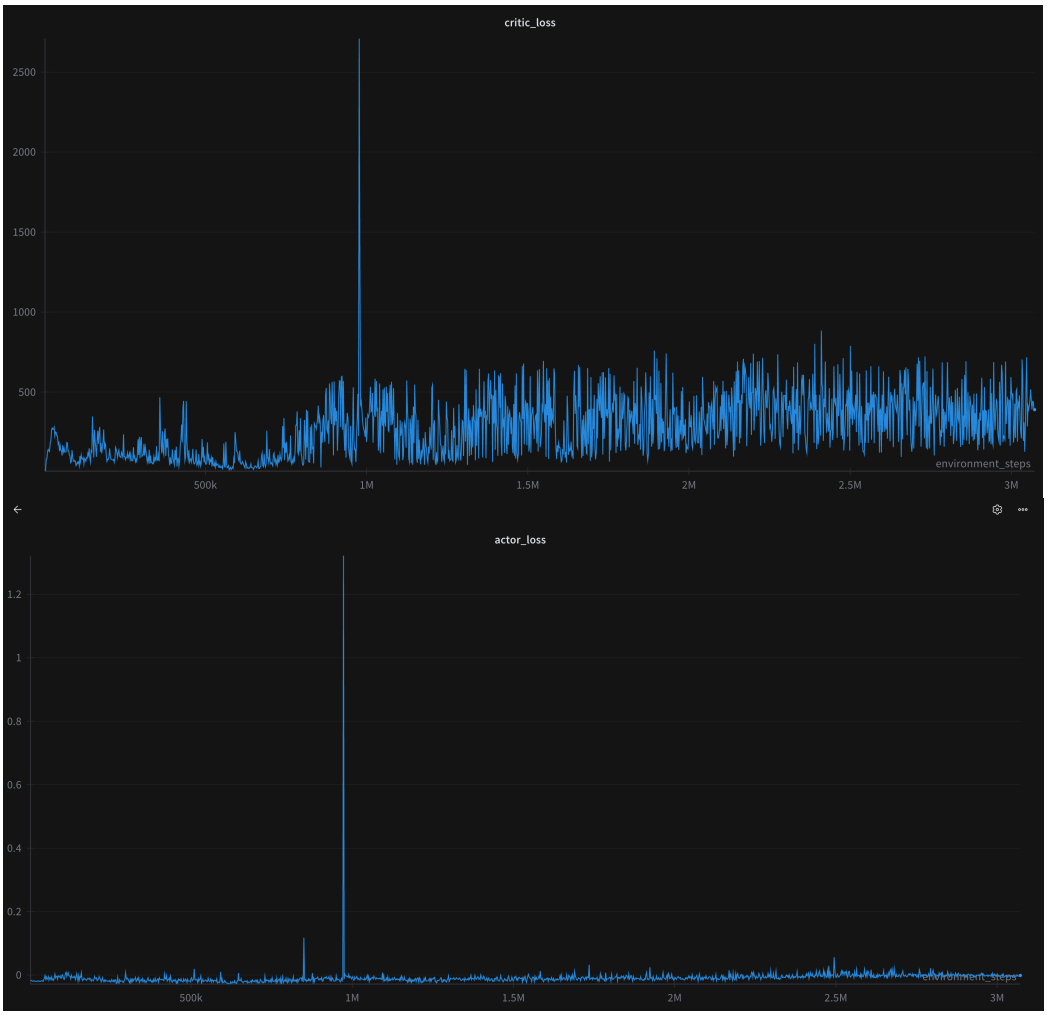

actor loss
200
0
-200
step
50k
100k
150k
200k
critic loss
500
400
300
200
100
step
50k
100k
150k
200k

Task 2


actor_loss
0
-0.01
-0.02
-0.03
total_steps
50k
100k
150k
200k


critic_loss
4000
3000
2000
1000
total_steps
50k
100k
150k
200k

Task 3


critic_loss
2500
2000
1500
1000
500
environment_steps
500k
1M
1.5M
2M
2.5M
3M
actor_loss
1.2
1
0.8
0.6
0.4
0.2
0
500k
1M
1.5M
2M
2.5M
3M

Evaluation screenshots

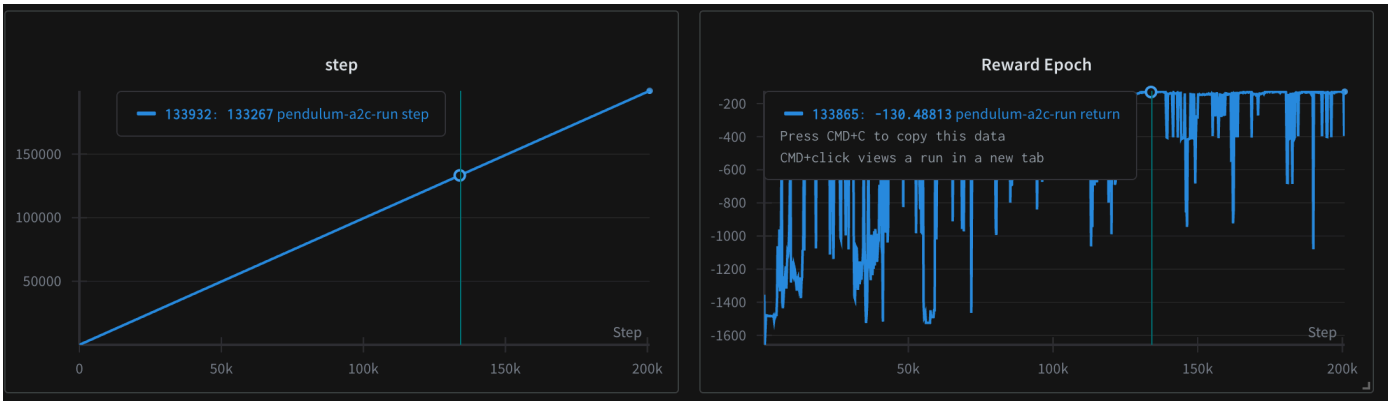
Task 1

我用的是 140k env step 存下來的 model

```
=== Video Recording Summary ===
Seed  10041 | Score:  -138.67 | Video saved in videos/seed_10041/
Seed  10042 | Score:  -132.06 | Video saved in videos/seed_10042/
Seed  10043 | Score:  -131.76 | Video saved in videos/seed_10043/
Seed  10044 | Score:  -263.71 | Video saved in videos/seed_10044/
Seed  10045 | Score:  -128.81 | Video saved in videos/seed_10045/
Seed  10046 | Score:    -4.28 | Video saved in videos/seed_10046/
Seed  10047 | Score:  -139.38 | Video saved in videos/seed_10047/
Seed  10048 | Score:    -5.34 | Video saved in videos/seed_10048/
Seed  10049 | Score:    -4.66 | Video saved in videos/seed_10049/
Seed  10050 | Score:  -268.51 | Video saved in videos/seed_10050/
Seed  10051 | Score:  -287.65 | Video saved in videos/seed_10051/
Seed  10052 | Score:    -4.35 | Video saved in videos/seed_10052/
Seed  10053 | Score:    -5.92 | Video saved in videos/seed_10053/
Seed  10054 | Score:  -134.54 | Video saved in videos/seed_10054/
Seed  10055 | Score:  -272.33 | Video saved in videos/seed_10055/
Seed  10056 | Score:  -138.72 | Video saved in videos/seed_10056/
Seed  10057 | Score:  -267.34 | Video saved in videos/seed_10057/
Seed  10058 | Score:  -270.75 | Video saved in videos/seed_10058/
Seed  10059 | Score:  -279.39 | Video saved in videos/seed_10059/
Seed  10060 | Score:    -5.05 | Video saved in videos/seed_10060/

Statistics:
  Mean Score: -144.16
  Std Score:  108.00
  Max Score:  -4.28
  Min Score:  -287.65

✓ SUCCESS: Average score -144.16 > -150
```



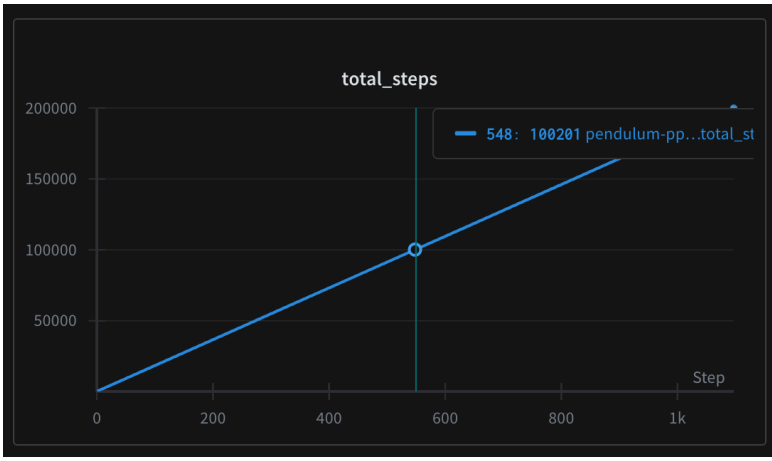
Task 2
我用的是 100k env step 存下的 model

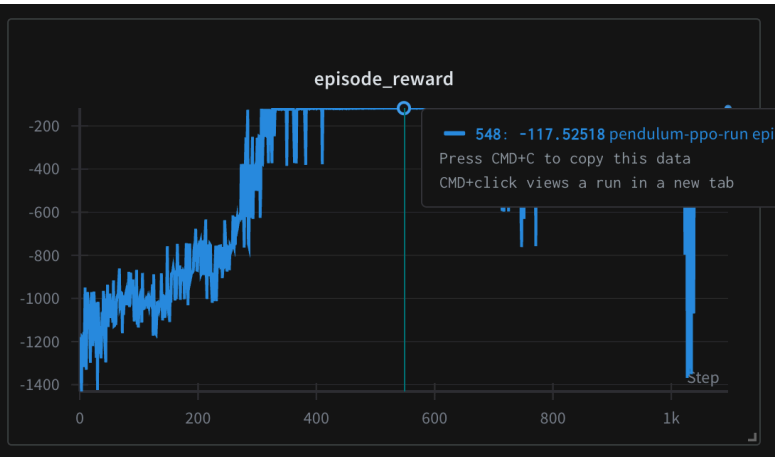
```
Statistics:
  Mean Score: -144.94
  Std Score:  92.70
  Max Score:  -0.29
  Min Score:  -381.65

✓ SUCCESS: Average score -144.94 > -150

=== Detailed Results for Best Window ===
Seed    553:  -117.60
Seed    554:  -381.65
Seed    555:  -239.76
Seed    556:  -118.46
Seed    557:  -117.81
Seed    558:  -121.59
Seed    559:  -115.03
Seed    560:    -1.98
Seed    561:  -117.51
Seed    562:  -122.51
Seed    563:  -250.14
Seed    564:  -120.56
Seed    565:    -0.29
Seed    566:  -357.08
Seed    567:  -120.73
Seed    568:  -114.73
Seed    569:  -118.59
Seed    570:  -119.43
Seed    571:  -118.44
Seed    572:  -124.94

Average: -144.94
 ~/Downloads                                  ✓ 14s Downloads base 20:37:49
  python infer_ppo_pendulum.py --checkpoint ./task2_result/ppo_pendulum_ep925_step185001.pt --mode con
secutive --window-size 20 --target-avg -150 --max-search 20 --start-seed 553 --render --record-consecut
ive
```





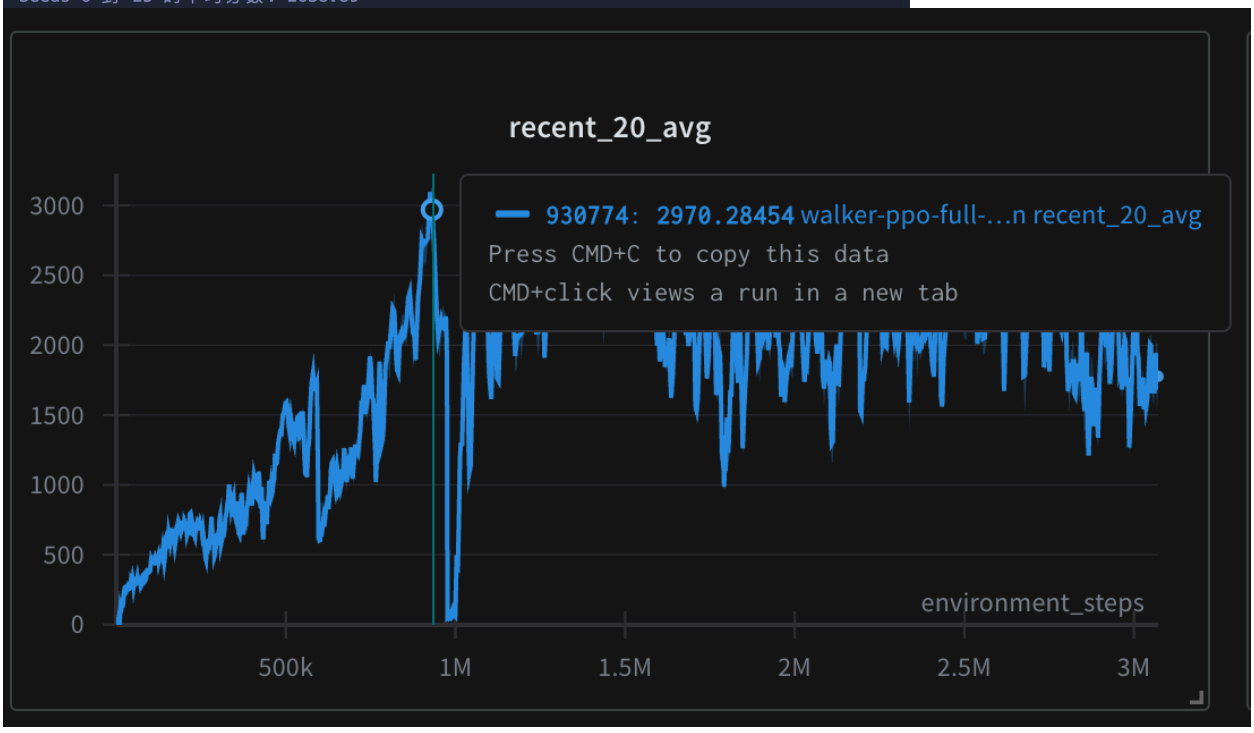
Task 3

我用的是約 900k 左右可以到 avg 2600 以上的 model checkpoint

```
deo` wrapper if this is not desired)
  logger.warn(
  Seed 6: 3666.86
/Users/minghsuan/Downloads/venv/lib/python3.11/site-packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The environment Walker2d-v4 is out of date. You should consider upgrading to version `v5`.
  logger.deprecation(
/Users/minghsuan/Downloads/venv/lib/python3.11/site-packages/gymnasium/wrappers/rendering.py:296: UserWarning: WARN: Overwriting existing videos at /Users/minghsuan/Downloads/result_task3 folder (try specifying a different `video_folder` for the `RecordVideo` wrapper if this is not desired)
  logger.warn(
  Seed 7: 1229.75
  Seed 8: 3698.49
  Seed 9: 1835.20
  Seed 10: 1405.79
  Seed 11: 1804.47
  Seed 12: 1743.99
  Seed 13: 2477.76
  Seed 14: 3672.23
  Seed 15: 1674.71
  Seed 16: 2251.27
  Seed 17: 3175.20
  Seed 18: 3756.18
  Seed 19: 3649.30
  Seed 20: 3749.56
  Seed 21: 2041.62
  Seed 22: 3568.98
  Seed 23: 3711.76
  Seed 24: 1916.28
  Seed 25: 1748.37

📊 錄製完成！
  - 平均分數：2638.89
  - 最高分數：3756.18
  - 最低分數：1229.75
  - 影片儲存於：result_task3/
  - 影片命名格式：seed-6_episode-0.mp4, seed-7_episode-0.mp4, ...

✅ 測試完成！
Seeds 6 到 25 的平均分數：2638.89
```

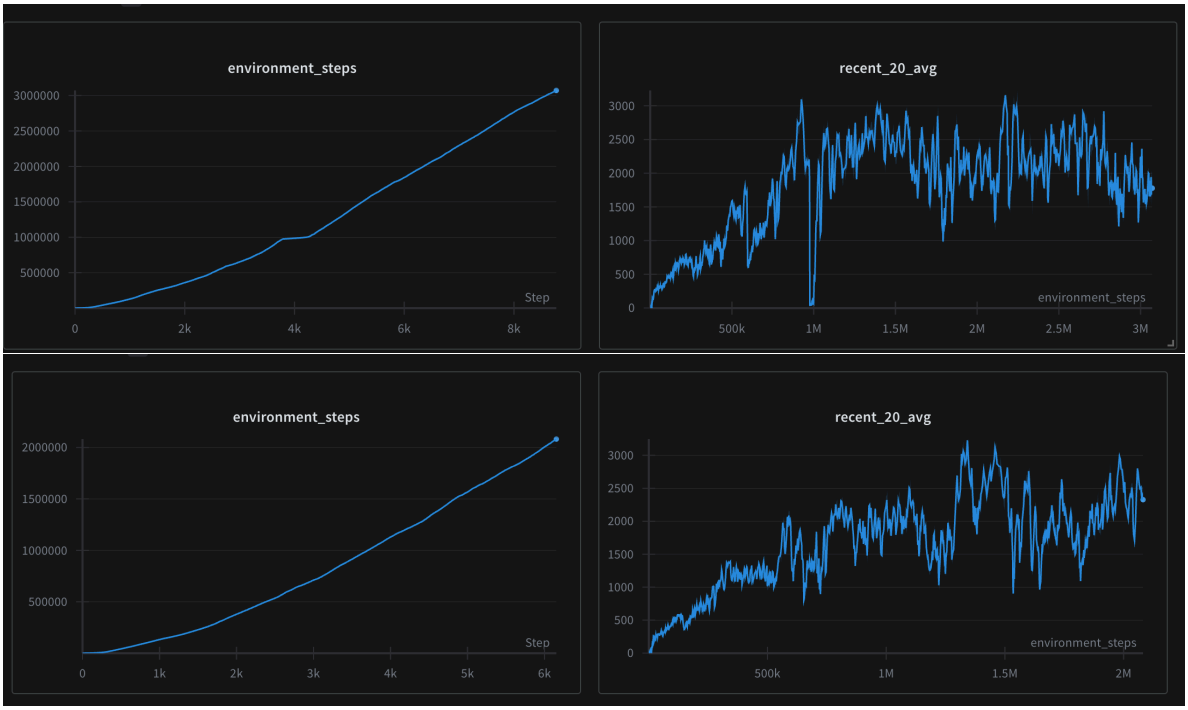


Compare the sample efficiency and training stability of A2C and PPO

從訓練曲線可以看到，PPO 在 sample efficiency 和 training stability 兩方面都明顯優於 A2C，Sample Efficiency 的 PPO 在 Pendulum 任務上大約在 56k environment steps 就達到 -150 的目標分數，而 A2C 需要 130k steps，是因為 PPO 的 clipped objective 避免了破壞性的 policy 更新並且 GAE 提供了更穩定的 advantage estimation 在 Multiple epochs per batch 提高了 data 利用效率，PPO 的學習曲線更加平滑，而 A2C 容易出現大幅震盪。這是因為 A2C 每步都更新，容易受到單個 bad sample 影響，而 PPO 的 clipping mechanism 和 batch update 提供了更好的穩定性。

Additional analysis

在 Task 3 我有去試過 3e-5(下面第一張)和 5e-5(下面第二張) learning rate 發現 5e-5 可以更快達到訓練目標而且他沒有 3e-5 那段很大的 loss。5e-5 learning rate 比 3e-5 更好我查過可能是因為它讓更新幅度讓 policy 持續改進複雜的步態模式，避免了過於保守的 lr 導致的"假收斂"和後期訓練崩潰。



Reference

https://blog.csdn.net/qq_44949041/article/details/130529916
https://hackmd.io/@shaoeChen/Bywb8YLKS/https%3A%2F%2Fhackmd.io%2F%40shaoeChen%2FHkH2hSKuS
https://www.youtube.com/watch?v=US8DFaAZcp4
https://www.slmt.tw/blog/2021/02/07/rl-note-1/
https://blog.csdn.net/wxc971231/article/details/121885755